

From Execution to Evolution: A Dual-Loop Agentic System for Long-Horizon Mobile Manipulation

Chenzhi Liu^{1,*} Zhang Yue^{1,*} Jiehong Lin^{1,*†}
Jianan Wang² Bo Wang¹ Zhongrui Wang^{3,‡} Xiaojuan Qi^{1,‡}

¹The University of Hong Kong ²Astribot

³Southern University of Science and Technology

*Equal contribution †Project Leader ‡Corresponding authors

Abstract: Long-horizon mobile manipulation presents significant challenges due to compounding execution errors and capacity interference between locomotion and arm control. While recent Vision-Language-Action models excel at short-horizon tasks, they lack the hierarchical reasoning required for multi-stage objectives. Furthermore, existing hierarchical agents suffer from rigid sub-task mapping, inflexible replanning, and a lack of continuous learning. To address these limitations, we introduce **MobiAgent**, a dual-loop agentic framework that bridges robust deployment execution and continuous policy evolution. During deployment, the *Inner Loop* decouples high-level reasoning from low-level control through highly composable *atomic skills*. It employs Vision-Language models for receding-horizon planning and visual reflection, dynamically composing skills to ensure robust error recovery. These skills are executed by specialized flow-matching experts that share a unified VLM backbone, maximizing reusability while mitigating capacity interference. Concurrently, the *Outer Loop* drives automated lifelong learning by autonomously segmenting and verifying deployment rollouts, clustering them to discover atomic skills, and continuously fine-tuning the skill library without human annotations. Evaluations across the simulated BEHAVIOR-1K benchmark and real-world tasks demonstrate that MobiAgent significantly outperforms state-of-the-art monolithic VLAs, achieving a +22.5% higher success rate in simulation. Furthermore, it demonstrates robust error recovery against critical failures and enables steady, annotation-free policy evolution via autonomous data recycling.

Keywords: Agentic System, Mobile Manipulation, Continuous Evolution

1 Introduction

Long-horizon mobile manipulation requires executing complex, multi-stage tasks to achieve global goals, necessitating the seamless interleaving of distinct behavioral modes including global navigation and precise prehension. This domain poses significant challenges due to compounding execution errors and the capacity interference inherent in jointly modeling disparate sensorimotor mappings for locomotion and arm control.

Recent Vision-Language-Action (VLA) models, notably OpenVLA [1], π_0 [2], and $\pi_{0.5}$ [3], have demonstrated exceptional proficiency in tabletop environments by directly mapping multimodal observations to continuous actions. Building on this foundation, systems like Mobi- π [4] and N2M [5] extend VLAs to mobile platforms, primarily optimizing the navigation-manipulation hand-off. Nevertheless, these monolithic architectures function predominantly as reactive policies tailored for short-horizon objectives. In long-horizon scenarios, they lack the hierarchical reasoning necessary

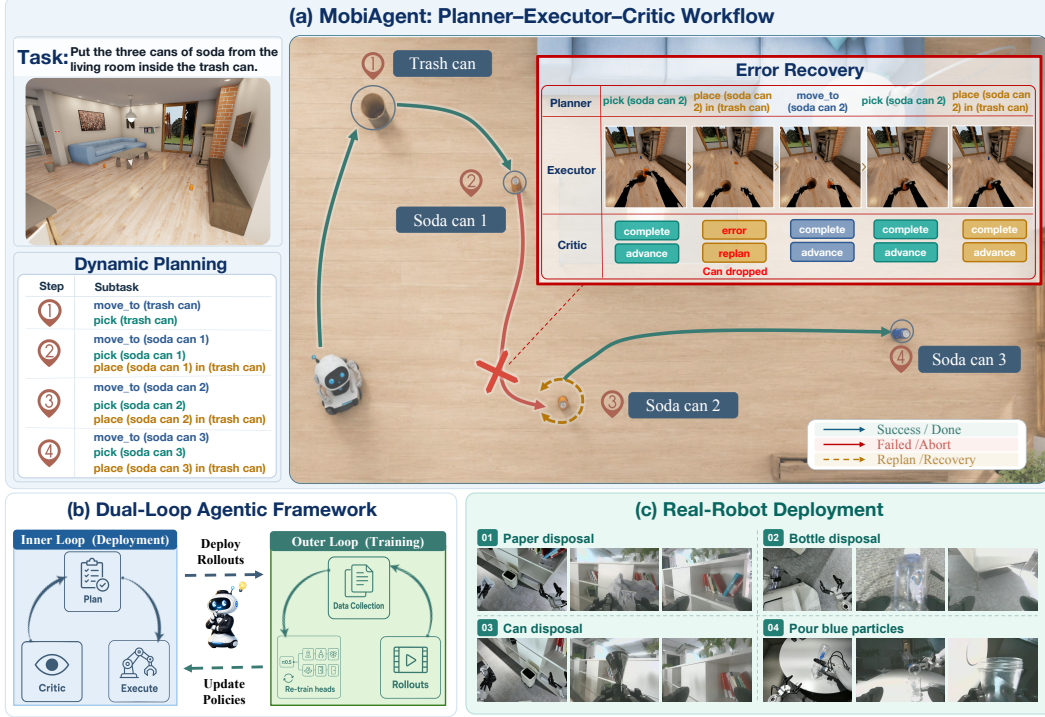


Figure 1: **Overview of MobiAgent.** MobiAgent solves a long-horizon picking-up-trash task by interleaving dynamic planning, skill-conditioned execution, and critic-guided recovery. The rollout trace illustrates a failed placement where the can drops outside the trash. In response, MobiAgent triggers replanning, guiding the robot to approach the displaced can, re-grasp it, and complete the task. This behavior is driven by our dual-loop framework (bottom left), which couples an online *Inner Loop* executing atomic skills with an offline *Outer Loop* that autonomously curates data for continuous policy evolution. The bottom right shows real-robot deployment on paper, bottle, and can disposal, as well as pouring blue particles into a cup.

for semantic task decomposition and struggle to maintain the long-term context required for closed-loop replanning, fundamentally limiting their autonomy and reliability.

To transcend short-horizon reactive policies, recent research explores hierarchical agentic frameworks that layer Large Language Models (LLMs) or Vision-Language Models (VLMs) over low-level controllers [6, 7, 8, 9, 10, 11, 12, 13], typically adopting a *plan-execute-verify* paradigm. Beyond their confinement to stationary manipulation, these architectures exhibit three critical bottlenecks: **1) Constrained Planning via Sub-task Mapping:** They typically assign an individual, isolated policy to each decomposed sub-task. High-level planning is then reduced to discrete selection from a predefined list rather than open-ended generation, inherently limiting scalability and increasing system overhead as the required sub-task repertoire expands. **2) Inflexible Verification and Replanning:** Stemming directly from this rigid policy mapping, dynamically replanning after a verification failure becomes highly cumbersome. This inflexibility is particularly fatal in mobile manipulation, where reflection and error recovery demand complex, continuous spatial reasoning (e.g., dynamic navigation, distance and viewpoint adjustments) prior to interaction. **3) Absence of Data Curation and Policy Evolution:** The vast majority of these frameworks operate open-loop regarding lifelong learning. While rare exceptions like RoboClaw [13] introduce automated data collection, the field still lacks a comprehensive pipeline for unconstrained data curation and continuous policy evolution.

To bridge the gap between robust deployment execution and continuous policy evolution, we propose **MobiAgent**, a dual-loop agentic system tailored for long-horizon mobile manipulation, as shown in

Fig. 1. MobiAgent decouples high-level reasoning from low-level control based on atomic skills, partitioning the system into a deployment-time Inner Loop and an offline-training Outer Loop.

To overcome the bottlenecks of constrained planning and inflexible replanning, the Inner Loop shifts the execution paradigm from rigid sub-task policies to highly composable *atomic skills* within a receding-horizon control flow. First, a **Task Planner** dynamically composes these skills on the fly. By replanning after completing each plan, it swiftly adapts to environmental changes and enables open-ended plan generation, bypassing the scalability limits of predefined sub-task lists. Next, by breaking down traditional sub-tasks into finer-grained actions, the **Skill Executor** employs skill-specific flow-matching experts built upon a shared VLM backbone. This semantic partitioning allows diverse sub-tasks sharing the same underlying action to reuse a single action expert, maximizing reusability while avoiding capacity interference. Finally, a **Reflection Critic** provides continuous visual verification to trigger flexible replanning. Operating at this granular level allows the Critic to explicitly manage the complex spatial reasoning essential for robust error recovery.

The Outer Loop drives the automated data curation and evolution of the skill library, eliminating the need for manual data curation. First, a VLM-powered **Data Curator** processes startup demonstrations and deployment rollouts to autonomously segment and verify trajectory clips with instruction captions. Next, an **Skill Generator** clusters these instructions to establish a discrete vocabulary of atomic skills, seamlessly standardizing new rollout data into this inventory. Finally, a **Skill Trainer** uses this curated data to instantiate and continuously fine-tune the skill policies.

To evaluate MobiAgent on long-horizon mobile manipulation, we conduct experiments across four tasks in the simulated BEHAVIOR-1K benchmark [14] and four real-world scenarios. Our evaluations demonstrate that MobiAgent significantly outperforms the state-of-the-art monolithic VLA, $\pi_{0.5}$ -TA [15], achieving a +22.5% higher average success rate in simulation. Furthermore, its critic-driven global replanning ensures robust error recovery against critical failures like object drops and pose misalignments. Finally, real-world deployments on the Atribot S1 validate that our Outer Loop enables steady, annotation-free policy evolution through autonomous data recycling.

2 Related Work

Vision-language-action (VLA) policies for short-horizon manipulation. Recent foundation models have popularized VLA policies that directly map language and visual observations to short-horizon robot actions [16, 1, 17, 2, 3, 18, 19, 20, 21]. To model these short-horizon action distributions, architectures typically adopt one of two paradigms: models like RT-2 [16] and OpenVLA [1] co-train web-scale VLMs to output discretized action tokens, whereas recent state-of-the-art approaches like π_0 [2] and $\pi_{0.5}$ [3] leverage flow matching to directly generate continuous action trajectories.

Hierarchical agentic frameworks for long-horizon tasks. Hierarchical frameworks typically layer LLMs or VLMs over low-level controllers using a plan-execute-verify paradigm [6, 7, 8, 9, 10, 11, 12, 22, 23, 24, 25, 26]. While effective, they often rely on fixed skill repertoires. For instance, CaP-X [27] composes predefined primitives but cannot learn underlying continuous sensorimotor behaviors. MobiAgent instead employs a unified, skill-conditioned VLA for native vision-based control. Additionally, existing error recovery mechanisms are often rigid; RoboClaw [13], for example, requires explicitly paired forward and reset policies. Finally, unlike most open-loop systems that leave failed behaviors unchanged, MobiAgent introduces a deployment-to-training pipeline to continuously refine its skills from curated rollouts.

Mobile manipulation systems. Current mobile manipulation systems [28, 29, 30, 31, 32, 33], including recent VLA extensions like Mobi- π [4] and N2M [5], excel at reactive control and navigation-manipulation hand-offs. However, they lack the closed-loop replanning, online verification, and deployment-driven refinement required for long-horizon execution. Furthermore, state-of-the-art approaches like $\pi_{0.5}$ -TA [15] (the 2025 BEHAVIOR Challenge winner) rely on isolated,

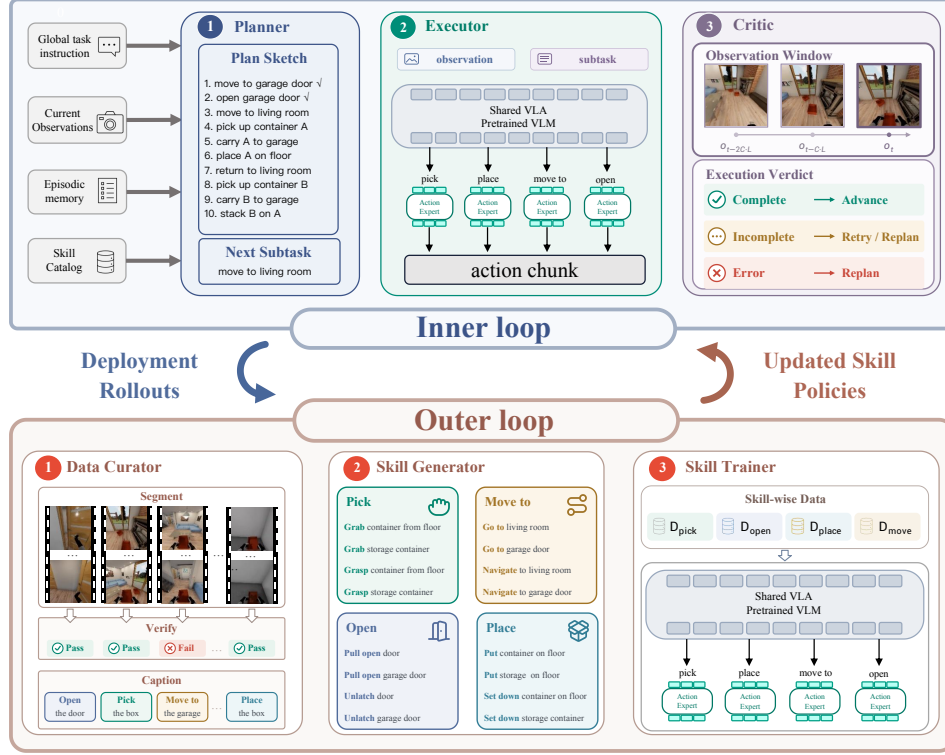


Figure 2: Illustration of the MobiAgent framework.

task-specific models. Their use of time-indexed stages forces distinct behaviors to share a single prediction head, causing capacity interference.

3 MobiAgent: A Dual-Loop Agentic System for Mobile Manipulation

Task Definition & Motivation. We address long-horizon mobile manipulation, where a robot must execute complex, multi-stage tasks specified by a global language instruction ℓ . At each step t , the robot maps multi-modal observations o_t to continuous base and manipulator actions $\mathbf{a}_t$, requiring the seamless interleaving of distinct behavioral modes like global navigation and precise prehension. Since monolithic end-to-end policies often suffer from compounding execution errors and capacity interference when jointly modeling disparate sensorimotor mappings, we propose **MobiAgent**. By decoupling high-level semantic reasoning from low-level sensorimotor control, our system enables dynamic task decomposition, spatial-aware failure recovery, and continuous offline skill evolution.

Dual-Loop Architecture. As shown in Fig. 2, MobiAgent employs a dual-loop agentic architecture to separate deployment-time execution from offline policy evolution:

- **Inner Loop (Deployment):** Governs closed-loop execution. It consists of a **Task Planner** that dynamically decomposes ℓ into actionable sub-tasks, a **Skill Executor** that maps sub-task instructions and real-time observations to low-level action chunks, and a **Reflection Critic** that evaluates outcomes to trigger progression, retries, or spatial-aware replanning.
- **Outer Loop (Offline Training):** Drives automated policy evolution. It ingests collected rollouts, utilizing a **Data Curator** for VLM-powered trajectory segmentation, a **Skill Generator** for semantic skill clustering, and a **Skill Trainer** for continuous policy updates.

3.1 The Inner Loop: Closed-Loop Agentic Execution

The inner loop performs closed-loop task execution at deployment time. As shown in Fig. 2, it repeatedly plans the next sub-task, executes a skill-conditioned action chunk, and evaluates the result. This process continues until the task succeeds or a fixed budget is exhausted. The overall control flow is not an open-loop decomposition followed by blind execution; instead, it is a recurrent plan–execute–verify loop grounded in current observations and recent failures.

3.1.1 Task Planner: Dynamic Atomic Skill Composition

The Task Planner, instantiated as a Vision-Language Model (VLM) Π_{plan} , shifts the execution paradigm from rigid sub-task mapping to the dynamic composition of composable *atomic skills*. Crucially, rather than generating a rigid, full-horizon plan from a predefined list of high-level sub-tasks, Π_{plan} operates in an open-ended, receding-horizon manner. It is invoked on the fly at specific decision steps k , triggered either by the successful completion of a preceding skill or by an execution failure. This formulation bypasses the scalability limits of static planning, allowing the agent to dynamically compose fine-grained skills to swiftly adapt to environmental changes and perturbations.

At each invocation step k , the Task Planner emits the next active sub-task conditioned on four inputs:

$$\ell_k, s_k = \Pi_{\text{plan}}(\ell, o_k, \mathcal{H}_k, \mathcal{S}).$$

Here, ℓ is the global task instruction, and o_k denotes the current multi-camera observation. $\mathcal{H}_k$ represents the episode memory, a running log of past sub-tasks, verification verdicts, and observation cues (e.g., spatial misalignments) generated by the Reflection Critic. $\mathcal{S}$ is the atomic skill catalog, exposed as an open-vocabulary registry. For the outputs, ℓ_k is a compact natural-language instruction for the immediate sub-task (e.g., “pick the red can”), and $s_k \in \mathcal{S}$ is a skill hint (e.g., `pick`). This hint serves as the critical interface for the Skill Executor to select the corresponding policy.

3.1.2 Skill Executor: Skill-Specific Action Generation

The Skill Executor, denoted as $\Pi_{\text{exec}} = \{\Pi_{\text{exec}}^{(s_k)}\}_{s_k \in \mathcal{S}}$, operates as a modular library of skill-specific policies dynamically indexed by the hint s_k . At each low-level control step t within the k -th sub-task, the Executor routes ℓ_k and the current multi-modal observation o_t to the selected policy, predicting a short-horizon action chunk of length L :

$$\mathbf{a}_{t:t+L} = \Pi_{\text{exec}}^{(s_k)}(\ell_k, o_t).$$

This semantic partitioning allows the system to specialize policies for distinct behavioral modes (e.g., locomotion vs. arm control) without suffering from the capacity interference.

We instantiate Π_{exec} using the $\pi_{0.5}$ Vision-Language-Action (VLA) model [3]. To maximize reusability while preserving optimization efficiency, we introduce a decoupling strategy: maintaining a shared VLM trunk π_{vlm} across all skills while instantiating independent flow-matching action experts $\pi_{\text{action}}^{(s_k)}$ for each skill. This is formulated as a function composition:

$$\mathbf{a}_{t:t+L} = \Pi_{\text{exec}}^{(s_k)}(\ell_k, o_t) = \pi_{\text{action}}^{(s_k)}(\pi_{\text{vlm}}(\ell_k, o_t)).$$

3.1.3 Reflection Critic: Continuous Evaluation and Feedback

To prevent compounding errors during long-horizon execution, we introduce a closed-loop feedback mechanism driven by the Reflection Critic Π_{critic} , which evaluates the execution outcomes to decide whether the active sub-task has been successfully completed, remains incomplete, or has failed.

Specifically, after the execution of every C action chunks (totaling $C \cdot L$ actions), Π_{critic} processes ℓ_k alongside a stride-sampled observation window $x_t = (o_{t-2C \cdot L}, o_{t-C \cdot L}, o_t)$. Relying exclusively on deployable observations, the Critic maps these inputs to a structured decision:

$$v_t, \phi_t, c_t = \Pi_{\text{critic}}(\ell_k, x_t).$$

The decision comprises a status verdict $v_t \in \{\text{complete}, \text{incomplete}, \text{error}\}$, a follow-up recommendation $\phi_t \in \{\text{next}, \text{retry}, \text{replan}\}$, and a set of observable cues c_t for offline auditing. To ensure deterministic control, the outputs are strictly constrained to a legal joint schema: 1) $v_t = \text{complete} \Rightarrow \phi_t = \text{next}$; 2) $v_t = \text{incomplete} \Rightarrow \phi_t \in \{\text{retry}, \text{replan}\}$; and 3) $v_t = \text{error} \Rightarrow \phi_t = \text{replan}$. Any schema violation is rejected, and v_t is set to **error**.

An orchestration controller Π_{Orch} governs the discrete state transitions by applying a deterministic policy over the validated (v_t, ϕ_t) tuples. Conditioned on a per-sub-task evaluation counter n_t and an upper bound $n_{\max}$, the transition dynamics are defined by the following mutually exclusive rules:

$$\Pi_{\text{Orch}}(v_t, \phi_t, n_t) = \begin{cases} \text{ADVANCE}, & v_t = \text{complete}, \\ \text{REPLAN}, & v_t = \text{error} \vee \phi_t = \text{replan}, \\ \text{RETRY}, & v_t = \text{incomplete} \wedge \phi_t = \text{retry} \wedge n_t < n_{\max}, \\ \text{REPLAN}, & v_t = \text{incomplete} \wedge \phi_t = \text{retry} \wedge n_t \geq n_{\max}. \end{cases}$$

Operationally, a **RETRY** command triggers continued execution of the active sub-task. Conversely, both **ADVANCE** and **REPLAN** halt the low-level execution and yield control back to the Planner. To facilitate spatial-aware replanning during this phase, the Critic’s evaluation logs and spatial cues c_t are persistently appended to the episodic memory $\mathcal{H}_k$.

3.2 The Outer Loop: Automated Agentic Training

The outer loop governs the offline training and continuous evolution of the skill library. It operates in two primary phases: first, *bootstrapping*, which initializes the system using domain-specific startup data (e.g., synthetic trajectories for simulation benchmarks or human-teleoperated demonstrations for real-world applications); and second, *continuous finetuning*, which runs offline after episode termination to curate deployment rollouts and iteratively refine the active policies.

To convert both startup data and deployment rollouts into actionable updates, trajectories are automatically processed via a three-stage pipeline (Fig. 2): 1) the **Data Curator** segments long trajectories into candidate clips paired with instruction captions, serving as atomic skill demonstrations; 2) the **Skill Generator** aggregates these captioned clips to formalize a discrete set of canonical atomic skills; and 3) the **Skill Trainer** utilizes the curated data to instantiate or update skill-wise policies.

3.2.1 Data Curator: VLM-Powered Trajectory Segmenting

The Data Curator transforms raw trajectories into candidate training clips with meaningful sub-task boundaries via an offline **segment-and-verify** pipeline (Fig. 2).

Segmenting. A video-capable VLM processes sampled frames from the full trajectory to propose temporal boundaries aligned with stage-level behaviors. For each segmented clip, the VLM generates a free-form action description. To ensure precise video clip captioning, the VLM employs a hierarchical reasoning process: it first determines whether the segment entails pure locomotion (without manipulation) and, if not, further infers the specific semantics of the manipulation types.

Verification. A separate VLM evaluates candidate clips against their proposed descriptions using a rigorous quality-control prompt. This verification process explicitly assesses temporal continuity, logical skill adjacency, and object-instruction consistency. Segments that accurately realize the intended behavior are accepted, whereas low-quality or boundary-misaligned clips are rejected and iteratively routed back for boundary refinement or instruction correction until they pass verification.

3.2.2 Skill Generator: Atomic Skill Clustering

Following trajectory curation, the Skill Generator standardizes the diverse, free-form instructions of verified clips into a compact, reusable inventory of atomic skills. For *startup data*, the system constructs the skill vocabulary from scratch through a three-step pipeline: first, an LLM performs semantic clustering on the instructions, dynamically determining the optimal number of clusters without predefined human priors; second, it assigns a canonical name to each cluster based on frequent textual patterns, enforced by a rigorous coverage check to resolve any omissions; third, the

Table 1: Success rates (%) on four BEHAVIOR-1K tasks compared to the $\pi_{0.5}$ -TA baseline [15].

Method	push-radio	dispose-trash	stack-storage	fetch-beer	MEAN
$\pi_{0.5}$ -TA [15]	60	40	60	10	42.5
MobiAgent (Ours)	70	60	70	60	65.0

LLM standardizes the original instructions by replacing diverse action verbs with these newly unified canonical names. For *rollout data*, the clustering and naming phases are bypassed; the LLM directly assigns the new clips to the previously established atomic skills and modifies their instructions accordingly. Ultimately, this adaptive approach transforms raw, heterogeneous fragments into a controlled skill vocabulary, providing a structured data substrate for downstream policy training.

3.2.3 Skill Trainer: Continuous Policy Evolution

The Skill Trainer completes the outer loop by translating the structured data into executable neural policies. During the bootstrapping phase with startup data, the Trainer dynamically instantiates dedicated action experts corresponding to the optimal number of canonical skills identified by the Generator. Sharing a common VLM trunk initialized from a pre-trained $\pi_{0.5}$ model (Sec. 3.1.2), these experts are subsequently trained via behavior cloning on their respective data clusters to establish the initial skill library. Conversely, during the continuous finetuning phase, the Trainer leverages newly curated deployment rollouts to directly update the existing policies, progressively refining their closed-loop robustness without expanding the model inventory.

4 Experiments

In this section, we conduct experiments to answer three core questions: (1) How does MobiAgent compare to state-of-the-art monolithic VLA models in long-horizon mobile manipulation (Sec. 4.1)? (2) Does MobiAgent effectively enable robust error recovery in dynamic environments (Sec. 4.2)? and (3) Can the Outer Loop drive steady, autonomous policy evolution (Sec. 4.3)? For additional results and implementation details, please refer to the supplementary material.

4.1 Long-Horizon Performance and Comparisons

We evaluate MobiAgent on four complex, diverse BEHAVIOR-1K tasks [14] via OmniGibson [34], including *push-radio* (single articulated-object interaction), *dispose-trash* (multi-object cross-room transport), *stack-storage* (ground placement and stacking), and *fetch-beer* (fetching with articulated closure). We compare against $\pi_{0.5}$ -TA [15], the 1st-place solution of the 2025 Challenge. $\pi_{0.5}$ -TA builds upon the base $\pi_{0.5}$ VLA model [3] by introducing correlated noise for flow matching to generate smoother actions, alongside learnable mixed-layer attention and stage tracking. Crucially, while $\pi_{0.5}$ -TA relies on a separate model trained for each specific task, MobiAgent utilizes skill policies shared across all tasks. We evaluate both systems under strictly matched conditions (10 trials per task, identical observations, embodiment, and timeouts). As shown in Table 1, MobiAgent achieves an average success rate of 65.0%, outperforming the task-specific $\pi_{0.5}$ -TA’s 42.5%. Notably, it yields a +10% gain on short articulated interactions (*push-radio*) and a substantial +50% gain on long-horizon fetch-with-closure tasks (*fetch-beer*). These results strongly validate that our modular role decomposition and closed-loop replanning are highly effective for solving complex, multi-stage household tasks compared to advanced monolithic VLAs.

4.2 Robustness and Error Recovery

To evaluate MobiAgent’s robustness in error recovery, we isolate the dynamic replanning loop driven by the Reflection Critic on the *dispose-trash* task in BEHAVIOR-1K. We compare a baseline that ignores global REPLAN signals (restricting the system to local retries within the active sub-task) against the full MobiAgent, which dynamically re-invokes the planner upon critic requests. We

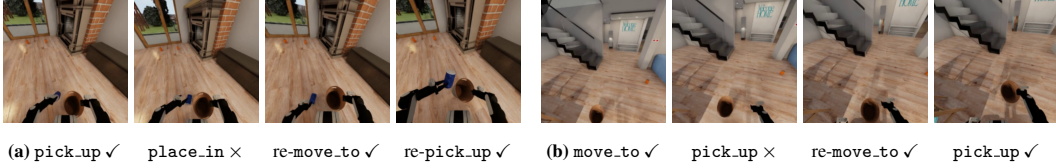


Figure 3: **Examples of robust error recovery.** MobiAgent overcomes execution failures through critic-driven global replanning: (a) generating new retrieval sub-tasks after an object is dropped, and (b) correcting the robot’s base pose before retrying the grasp.

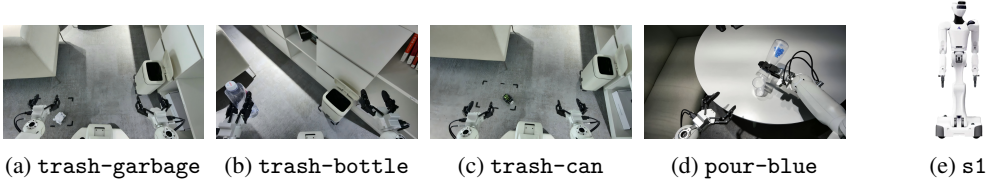


Figure 4: (a-d) Representative scene snapshots for real-world tasks and (e) the used Astribot S1 robot.

observe a success rate increase from 40% to 60%, confirming that the critic effectively identifies complex failures requiring global replanning. As visualized in Fig. 3, MobiAgent successfully overcomes critical failure modes such as object drops and pose misalignments. For instance, if an object is dropped during placement (Row a), the Critic triggers a global replan to generate new retrieval sub-tasks; similarly, a failed grasp due to a poor base pose (Row b) prompts an updated approach sub-task to correct the alignment. Ultimately, integrating granular visual verification with open-ended replanning enables MobiAgent to ensure robust execution in dynamic environments.

4.3 Automated Policy Evolution

To evaluate the Outer Loop’s capacity for automated evolution, we deploy MobiAgent on Astribot S1 [35], a bimanual mobile manipulator equipped with multi-view cameras and proprioceptive sensors operating at 30 Hz. We evaluate the system on four long-horizon tasks (Fig. 4) that share skill-specific policies: disposing of various objects into a trash can (trash-garbage, trash-bottle, trash-can) and pouring blue particles from a bottle into a cup (pour-blue). We bootstrap the initial policies (Round 1) using 100 demonstrations per task. To drive self-improvement, we execute an iterative loop: MobiAgent is deployed autonomously across all four tasks, and the resulting roll-outs are automatically curated by the Outer Loop. In each round, approximately 20 successful new trajectories per task are integrated to further refine the policies. As shown in Table 2, MobiAgent exhibits steady performance gains after two such rounds, confirming that our Outer Loop effectively recycles deployment data to drive continuous, annotation-free policy evolution.

5 Conclusion and Limitations

In this work, we presented MobiAgent, a dual-loop agentic framework designed to overcome the bottlenecks of long-horizon mobile manipulation. By shifting from rigid sub-task mapping to the dynamic composition of atomic skills, the Inner Loop enables open-ended, receding-horizon execution and critic-driven error recovery. Concurrently, the Outer Loop establishes an automated pipeline for continuous data curation and annotation-free policy evolution. Empirical evaluations demonstrate that MobiAgent significantly outperforms the state-of-the-art $\pi_{0.5}$ -TA baseline by 22.5% in simulated BEHAVIOR-1K environments, with real-world deployments on the Astribot S1 further validating its robust closed-loop execution and steady lifelong learning capabilities.

Limitations. While MobiAgent demonstrates robust performance, we identify a few areas for straightforward refinement in future work: 1) **Plug-and-Play Skill Expansion:** Currently, integrat-

Table 2: Success rates (%) of automated policy evolution on real-world experiments. Subsequent rounds refine the skill policies using autonomously curated deployment data.

Round	trash-garbage	trash-bottle	trash-can	pour-blue	MEAN
Round 1 (Bootstrap)	40	50	20	20	32.5
Round 2 (Iter 1)	40	60	30	30	40.0 (+7.5)
Round 3 (Iter 2)	50	60	70	50	57.5 (+17.5)

ing newly discovered skills into the shared VLA backbone can occasionally introduce capacity interference with prior behaviors. However, this is a well-studied phenomenon in multi-task learning and can be readily resolved in future iterations by adopting modular architectures, such as lightweight adapters (e.g., LoRA) or standard continual-learning objectives, enabling seamless, non-destructive skill integration without altering the core framework. 2) **Inner-Loop Edge Cases:** We observe minor execution artifacts in edge cases, including occasional *pose-correction churn* from navigation offsets and *critic verdict drift* during visual occlusions. As our framework is inherently data-driven, these isolated failures can be naturally mitigated by curating targeted failure-mode datasets and applying routine fine-tuning. 3) **Evaluation Scope:** Having validated the system on complex BEHAVIOR-1K tasks and real-world scenarios, a natural next step is extending the evaluation to broader task families and diverse robot embodiments to further showcase its general-purpose scalability.

Acknowledgments

The work has been supported by Hong Kong Research Grant Council - General Research Fund Scheme (Grant No. 17202422, 17212923, 17215025) Theme-based Research (Grant No.T45-701/22-R), and Strategic Topics Grant (Grant No.STG3/E-605/25-N). Part of the described research work is conducted in the JC STEM Lab of Robotics for Soft Materials funded by The Hong Kong Jockey Club Charities Trust.

References

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024.
- [2] Physical Intelligence. π_0 : A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*, 2025.
- [3] Physical Intelligence. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Conference on Robot Learning (CoRL)*, 2025.
- [4] J. Yang, I. Huang, B. Vu, M. Bajracharya, R. Antonova, and J. Bohg. Mobi- π : Mobilizing your robot learning policy. In *Conference on Robot Learning (CoRL)*, 2025.
- [5] K. Chai, H. Lee, and J. J. Lim. N2M: Bridging navigation and manipulation by learning pose preference from rollout. *arXiv preprint arXiv:2509.18671*, 2025.
- [6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [7] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [8] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023.
- [9] X. Li, M. Zhang, Y. Geng, H. Geng, Y. Long, Y. Shen, R. Zhang, J. Liu, and H. Dong. ManipLLM: Embodied multimodal large language model for object-centric robotic manipulation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [10] D. Honerkamp, M. Büchner, F. Despinoy, T. Welschhold, and A. Valada. Language models as zero-shot trajectory generators for long-horizon mobile manipulation. *IEEE Robotics and Automation Letters (RA-L)*, 2024.
- [11] L. X. Shi, B. Ichter, M. Equi, L. Ke, K. Pertsch, Q. Vuong, J. Tanner, A. Walling, H. Wang, N. Fusai, A. Li-Bell, D. Driess, L. Groom, S. Levine, and C. Finn. Hi Robot: Open-ended instruction following with hierarchical vision-language-action models. In *International Conference on Machine Learning (ICML)*, 2025.
- [12] H. Zhao, J. Zhang, W. Song, P. Ding, and D. Wang. VLA²: Empowering vision-language-action models with an agentic framework for unseen concept manipulation. *arXiv preprint arXiv:2510.14902*, 2025.
- [13] R. Li, Y. Zhou, Y. Zhu, K. Chen, J. Wang, S. Wang, K. Hu, M. Yu, B. Jiang, Z. Su, J. Ma, X. He, Y. Shen, Y. Yang, G. Ren, M. Yao, W. Wang, and Y. Mu. RoboClaw: An agentic framework for scalable long-horizon robotic tasks. *arXiv preprint arXiv:2603.11558*, 2026.

- [14] C. Li, R. Zhang, J. Wong, C. Gokmen, S. Srivastava, R. Martín-Martín, C. Wang, G. Levine, M. Lingelbach, J. Sun, et al. BEHAVIOR-1K: A benchmark for embodied AI with 1,000 everyday activities and realistic simulation. In *Conference on Robot Learning (CoRL)*, 2022.
- [15] I. Larchenko, G. Zarin, and A. Karnatak. Task adaptation of vision-language-action model: 1st place solution for the 2025 BEHAVIOR challenge. *arXiv preprint arXiv:2512.06951*, 2025.
- [16] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.
- [17] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, C. Xu, J. Luo, T. Kreiman, Y. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- [18] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, M. Liao, F. Wei, Y. Deng, S. Xu, Y. Zhang, X. Wang, B. Liu, J. Fu, J. Bao, D. Chen, Y. Shi, J. Yang, and B. Guo. CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024.
- [19] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, and X. Li. SpatialVLA: Exploring spatial representations for visual-language-action model. In *Robotics: Science and Systems (RSS)*, 2025.
- [20] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [21] X. Li, M. Liu, H. Zhang, C. Yu, J. Xu, H. Wu, C. Cheang, Y. Jing, W. Zhang, H. Liu, H. Li, and T. Kong. Vision-language foundation models as effective robot imitators. In *International Conference on Learning Representations (ICLR)*, 2024.
- [22] Y. Pang, B. Zhou, C. Li, X. Wang, S. Xu, D.-B. Wang, M.-L. Zhang, and S. Di. Sci-VLA: Agentic VLA inference plugin for long-horizon tasks in scientific experiments. *arXiv preprint arXiv:2602.09430*, 2026.
- [23] Y. Guo, Y.-J. Wang, L. Zha, and J. Chen. DoReMi: Grounding language model by detecting and recovering from plan-execution misalignment. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.
- [24] Z. Liu, A. Bahety, and S. Song. REFLECT: Summarizing robot experiences for failure explanation and correction. In *Conference on Robot Learning (CoRL)*, 2023.
- [25] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. I. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandekar, and Y. Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2025.
- [26] Z. Liu, X. Ning, Z. Hu, X. Xie, W. Li, Z. Tang, C. Wang, Z. Yang, H. Wang, Y. Liu, and Z. Pu. Goal2Skill: Long-horizon manipulation with adaptive planning and reflection. *arXiv preprint arXiv:2604.13942*, 2026.
- [27] M. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, G. Wang, F.-F. Li, G. Shi, J. Wu, S. Sastry, Y. Zhu, K. Goldberg, and L. Fan. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026.
- [28] J. Wu, R. Antonova, A. Kan, M. Lepert, A. Zeng, S. Song, J. Bohg, S. Rusinkiewicz, and T. Funkhouser. TidyBot: Personalized robot assistance with large language models. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.

- [29] O. Lemke, Z. Bauer, R. Zurbrügg, M. Pollefeys, F. Engelmann, and H. Blum. Spot-compose: A framework for open-vocabulary object retrieval and drawer manipulation in point clouds. In *IEEE International Conference on Robotics and Automation (ICRA) Workshops*, 2024.
- [30] Z. Fu, T. Z. Zhao, and C. Finn. Mobile ALOHA: Learning bimanual mobile manipulation with low-cost whole-body teleoperation. In *Conference on Robot Learning (CoRL)*, 2024.
- [31] Z. Fu, Q. Zhao, Q. Wu, G. Wetzstein, and C. Finn. HumanPlus: Humanoid shadowing and imitation from humans. In *Conference on Robot Learning (CoRL)*, 2024.
- [32] P. Liu, Y. Orru, J. Vakil, C. Paxton, N. M. M. Shafiullah, and L. Pinto. OK-Robot: What really matters in integrating open-knowledge models for robotics. In *Robotics: Science and Systems (RSS)*, 2024.
- [33] S. Yenamandra, A. Ramachandran, K. Yadav, A. Wang, M. Khanna, T. Gervet, T.-Y. Yang, V. Jain, A. W. Clegg, J. Turner, Z. Kira, M. Savva, A. Chang, D. S. Chaplot, D. Batra, R. Motlaghi, Y. Bisk, and C. Paxton. HomeRobot: Open-vocabulary mobile manipulation. In *Conference on Robot Learning (CoRL)*, 2023.
- [34] C. Li, F. Xia, R. Martín-Martín, M. Lingelbach, S. Srivastava, B. Shen, K. Vainio, C. Gokmen, G. Dharan, T. Jain, A. Kurenkov, C. K. Liu, H. Gweon, J. Wu, L. Fei-Fei, and S. Savarese. iGibson 2.0: Object-centric simulation for robot learning of everyday household tasks. In *Conference on Robot Learning (CoRL)*, 2021.
- [35] G. Gao, J. Wang, J. Zuo, J. Jiang, J. Zhang, X. Zeng, Y. Zhu, L. Ma, K. Chen, M. Sheng, R. Zhang, and Z. An. Towards human-level intelligence via human-like whole-body manipulation. *arXiv preprint arXiv:2507.17141*, 2025.
- [36] OpenAI. Introducing GPT-5.4, 2026. URL <https://openai.com/index/introducing-gpt-5-4/>.
- [37] Google DeepMind. Gemini 3.1 Pro: A smarter model for your most complex tasks, 2026. URL <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/>.
- [38] M. Sundermeyer, A. Mousavian, R. Triebel, and D. Fox. Contact-GraspNet: Efficient 6-DoF grasp generation in cluttered scenes. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

Contents

A	LLM/VLM Configurations and Prompt Design	14
A.1	Task Planner	14
A.2	Reflection Critic	15
A.3	Data Curator - Segmenting	16
A.4	Data Curator - Verification	16
A.5	Skill Generator	16
B	Implementation Details of Simulated Experiments	17
B.1	Simulation Environment and Embodiment	17
B.2	Evaluated Tasks and Subtask Sequences	17
B.3	Skill Executor Policy Training	18
C	Implementation Details of Real-Robot Experiments	19
C.1	Real-Robot Environment and Embodiment	19
C.2	Real-Robot Evaluated Tasks and Subtask Sequences	19
C.3	Skill Executor Policy Training	20
D	Additional Quantitative Results and Analysis	20
D.1	Per-skill Policy Heads inside the Skill Executor	20
D.2	Data-collection Wall-clock Cost versus Manual Annotation	21
D.3	Per-episode Runtime	21
D.4	Per-subtask Success Rate	22
E	Failure Modes and Analysis	22
E.1	FM1: Reflection Critic False Positives	23
E.2	FM2: Bimanual-Coordination Failures	23
E.3	FM3: Inter-skill Transition and Pose Mismatch	23
E.4	FM4: Low-level Perceptual Limits	25
F	Comparison with Other Agentic Frameworks	25
F.1	Comparison with CaP-X	25
F.2	Comparison with RoboClaw	26

A LLM/VLM Configurations and Prompt Design

Table S1 details the default foundation models powering each module within the MobiAgent framework. While most components utilize GPT-5.4 [36], the Data Curator specifically relies on Gemini-3.1-Pro [37] for trajectory segmentation, driven by the need to accurately localize skill boundaries over continuous video. Because the GPT-5.4 endpoint does not natively support continuous video input, approximating the recording via uniformly sampled frames markedly degrades temporal boundary localization. In contrast, Gemini-3.1-Pro accepts native video input, ensuring precise temporal segmentation.

Table S1: Default LLM/VLM models used in our MobiAgent.

Agent	Model
Task Planner	GPT-5.4
Reflection Critic	GPT-5.4
Data Curator (segmentation)	Gemini-3.1-Pro
Data Curator (verification)	GPT-5.4
Skill Generator (aggregation)	GPT-5.4
Skill Trainer	GPT-5.4

For the prompt-driven modules, we employ a unified, framework-level prompt design. Crucially, these prompts are strictly task-agnostic: they specify the canonical skill set (i.e., the `stage_hint` used for policy routing) and the verb-frame skill templates, but explicitly exclude any specific task, room, or object names from the evaluation suite. Instance-specific context is dynamically injected solely through the goal, history, and observation fields. This zero-shot design ensures that the exact same prompts are reused without modification across all tasks.

In the following subsections, we provide the complete prompt templates for all LLMs and VLMs utilized across both the inner loop (Task Planner and Reflection Critic) and the outer loop (Data Curator for segmentation and verification, and Skill Generator for skill aggregation). Furthermore, all trailing JSON schemas are strictly enforced via constrained decoding to guarantee reliably formatted outputs.

A.1 Task Planner

System prompt: choose the next subtask

You are an online planner for a mobile-manipulation robot. On each call you choose the *single next* subtask, given the goal, the history of subtasks already finished, and the current head-camera image. You do not plan the whole sequence up front---you are re-invoked after each subtask is judged complete, so you can react to a displaced object, an already-open door, or a dropped item.

CANONICAL SKILLS (stage_hint, one per subtask)

move_to, pick_up, place_in, place_on, open, close, ...

(an open-ended catalogue---additional skills are supported simply by following the same skill-string template below).

SKILL TEMPLATE (the wire prompt is the bare skill string only)

move to <target>

pick up <object> from <source>

place <object> on <surface> / place <object> in <receptacle>

open <X> / close <X> / press <X>

INPUTS

global.goal: <string>

history: <finished subtasks; each gives completed/failed, chunks spent, and the Critic's cues>

image: <current head-camera frame>

OUTPUT (JSON)

```
{ "id": <string>,
  "plan_sketch": [<remaining steps, natural language>],
```

```

"prompt": <verb-frame realisation of the next step>,
"stage_hint": <one canonical skill>,
"target_object": <string|null>,
"rationale": <one short sentence> }

```

RULES

1. Emit only the next step; the orchestrator decides termination from the task's success check.
2. Use bare lowercase noun phrases taken verbatim from the goal, always singular even for repeated instances, with no ordinal or 'while carrying' suffix.
3. A move_to target is the object or surface the next manipulation acts on---never a room or area name.
4. On a replan, re-decide the next step from the current post-failure scene rather than resuming the old plan.

A.2 Reflection Critic

System prompt: judge whether the active subtask is complete

You judge whether the active subtask is complete from the camera frames captured over the last few execution chunks. Grade the most recent frame; use the earlier frames to read motion---is the base approaching, the arm rising, the panel rotating, or is it stuck / oscillating?

INPUTS

```

subtask: { prompt, stage_hint, target_object }
frames: <multi-camera composites (head + wrists), oldest→newest>

```

SUCCESS CONDITIONS (generalised, by skill)

move_to: the target is within the robot's reach zone at its base.
pick_up: a real grasp/lift signal---the object rises with the gripper, or the gripper closes on the object's body.
place_in/place_on: the object is at rest in/on the named receptacle/surface, gripper released.
open/close: the named panel has visibly changed state and come to rest.
press: the control's indicator has visibly changed state.
any other skill: judged by the same principle---its own observable success signal, i.e. the visible state change that defines its completion.

OUTPUT (JSON)

```

{ "verdict": "complete" | "incomplete" | "error",
  "followup": "next" | "retry" | "replan",
  "cues": [<observable cues for offline auditing, one labelled per relevant camera>] }

```

RULES

1. Ground every verdict in what is actually visible; never fabricate a position, colour, or motion you cannot see.
2. Be conservative---default to incomplete under any ambiguity, since a false complete makes the next subtask act on empty air.
3. Use error (followup replan) only for an observably broken state---a stranded arm, or a clearly different object at the reach position, which you note among the cues.
4. Return only a legal (verdict, followup) pair---complete→next; incomplete→retry or replan; error→replan. Any other combination is rejected by the orchestrator.

A.3 Data Curator - Segmenting

System prompt: segment a demonstration into atomic-skill clips

Segment a robot-demonstration recording into contiguous atomic-skill clips. Each clip is exactly one atomic intent: either a TRAVEL (the base navigating toward a target) or a STATE CHANGE (a manipulation that alters an object's pose or an articulated part's state); the two are mutually exclusive within a clip.

INPUTS

recording: <composite head + wrist views; each frame carries a burned-in t=...s playback-time label and the base-velocity readout>

OUTPUT (JSON)

```
{ "skill_timeline": [
  { "segment_id": <string>,
    "start_time_sec": <float>, "end_time_sec": <float>,
    "skill_description": <one-line instruction for this clip> },
  ... ] }
```

RULES

1. Coverage: start at 0.0, consecutive clips touch exactly, end at the last visible t=...s; copy boundaries from the visible time labels.
2. Every gripper acquire→release cycle on a carried object is split into a separate acquisition clip and a separate placement clip, with a navigation clip inserted whenever the base travels between them---never one compound clip.
3. Pre-arrival travel is its own navigation clip; absorb brief approach / align / idle intervals into the adjacent state-change clip.
4. No hallucinated state changes: label a state change only when the change is actually observed, and forbid prep-only labels (adjust, reposition, wait).

A.4 Data Curator - Verification

System prompt: check a proposed skill segmentation for consistency

You receive a proposed segmentation of one demonstration---an ordered list of skill segments, each described by a skill description, a stage hint, and a target object---and decide whether the segmentation is internally consistent and well-formed for the stated goal. You reason over the textual segmentation only; you do not watch the recording.

INPUTS

goal: <string>

segments: [<ordered list of { skill_description, stage_hint, target_object }>]

OUTPUT (JSON)

```
{ "valid": true | false, "reason": <one sentence, ≤ 25 words> }
```

RULES

1. valid=true only if every segment is a well-formed canonical skill on a plausible target and the ordered sequence is coherent for the goal.
2. valid=false on a malformed segmentation---an incoherent or contradictory ordering, an impossible step, or redundant segments that cannot co-occur.
3. Be conservative---prefer valid=false when the segmentation is ambiguous or under-specified.

A.5 Skill Generator

System prompt: cluster skill descriptions into canonical skills

You receive the free-form skill descriptions produced by segmentation across many demonstrations. Cluster them by action intent, then give each cluster one reusable canonical skill name.

INPUTS

items: [{ item_id, description }, ...] (a flat list, not pre-clustered)

OUTPUT (JSON)

```
{ "clusters": [
  { "canonical_name": <short action-intent label>,
    "activity_type": "navigation" | "manipulation",
    "item_ids": [<ids>] },
  ...] }
```

RULES

1. Cluster by the action performed, not by the task-specific object, room, or support surface.
2. canonical_name is a short reusable action-intent label.
3. Prefer general categories over fine-grained variants---do not split placing (stack / drop-on / rest-on), acquiring (fetch / retrieve / extract), or actuating (pull-open / swing-shut) into separate names.
4. Fold preparation / contact / completion phases into the state-changing skill they support; never leave them as standalone skills.
5. Every item_id appears in exactly one cluster.

B Implementation Details of Simulated Experiments

B.1 Simulation Environment and Embodiment

Our simulation experiments are conducted in the OmniGibson simulator using the BEHAVIOR-1K benchmark [14]. We adopt the Galaxea R1 Pro, the default embodiment of the 2025 BEHAVIOR Challenge. This wheeled humanoid features an omnidirectional mobile base, a liftable torso, and two 6-DoF arms, making it ideal for tasks requiring extensive end-effector reachability, bimanual coordination, and stable navigation.

The policy receives onboard RGB observations rendered at 224×224 resolution and outputs a 32-dimensional action chunk. Task success is rigorously evaluated using the BDDL goal predicates associated with each activity (Table S2). For evaluation, every (task, method) pair is averaged across 10 random seeds under a fixed scene-reset protocol. Crucially, rather than utilizing long monolithic trajectories, per-skill training clips are extracted from the challenge’s teleoperated demonstrations via our Data Curator pipeline.

B.2 Evaluated Tasks and Subtask Sequences

Table S2: Task instructions and main-paper task names for the four evaluated BEHAVIOR-1K tasks.

Task ID	Task name	Instruction
task-0000	push-radio	“Turn on the radio on the coffee table.”
task-0001	dispose-trash	“Put the three cans of soda from the living room inside the trash can in the kitchen.”
task-0016	stack-storage	“Move the two storage containers from the living room to the garage. In the garage, place one container on the floor and stack the other container on top of it (either order is fine).”
task-0017	fetch-beer	“Bring two beer bottles from the refrigerator to the coffee table, then close the refrigerator.”

Table S2 details the natural-language instructions for the four evaluated BEHAVIOR-1K tasks, alongside their abbreviated names used throughout the main text. During an episode, these instruction strings are passed directly into the *global.goal* field of the Task Planner without modification. Subsequently, Table S3 outlines the representative per-task subtask sequences used to train the per-skill policies of the Skill Executor in simulation. Each line corresponds to a single segment tagged with one of the canonical skills (*move_to*, *pick_up*, *place_in*, *place_on*, *open*, *close*). Because the segmentation is highly consistent across episodes, these sequences accurately represent the structural breakdown of each task.

Table S3: Representative verbatim subtask sequences for the four BEHAVIOR-1K tasks.

Task	Subtask Sequence (verbatim)
push-radio	1. move to radio 2. pick up radio from coffee table 3. press radio 4. place radio on coffee table
dispose-trash	1. move to trash can 2. pick up trash can from floor 3. move to can of soda 4. pick up can of soda from floor 5. place can of soda in trash can 6. move to can of soda 7. pick up can of soda from floor 8. place can of soda in trash can 9. move to can of soda 10. pick up can of soda from floor 11. place can of soda in trash can 12. place trash can on floor
stack-storage	1. move to door 2. open door 3. move to storage container 4. pick up storage container from floor 5. move to floor 6. place storage container on floor 7. move to storage container 8. pick up storage container from floor 9. move to floor 10. place storage container on storage container
fetch-beer	1. move to refrigerator 2. open refrigerator 3. pick up beer bottles from refrigerator 4. move to coffee table 5. place beer bottles on coffee table 6. move to refrigerator 7. close refrigerator

Table S4: Training hyperparameters for the per-skill $\pi_{0.5}$ fine-tuning in simulation (BEHAVIOR-1K).

Setting	Simulation
Precision	bfloat16
Batch size	480
Training steps	50,000
Warmup steps	1,000
Learning rate	1×10^{-4}

B.3 Skill Executor Policy Training

The Skill Executor is instantiated as a single $\pi_{0.5}$ [3] model. This model features a shared PaliGemma (gemma_2b) backbone and routes to distinct action experts for each canonical skill, where each expert is a gemma_300m head. All experts are initialized from the publicly available $\pi_{0.5}$ base checkpoint and subsequently fine-tuned on the skill-specific segment clips generated by the Data Curator pipeline.

Specifically, the six BEHAVIOR-1K skills (move_to, pick_up, place_in, place_on, open, close) are fine-tuned jointly on $8 \times$ A100 GPUs using 8-way Fully Sharded Data Parallel (FSDP). We apply a global action normalizer, and the batch shares for each skill are allocated proportionally to their respective dataset sizes. Table S4 summarizes the complete training configurations.

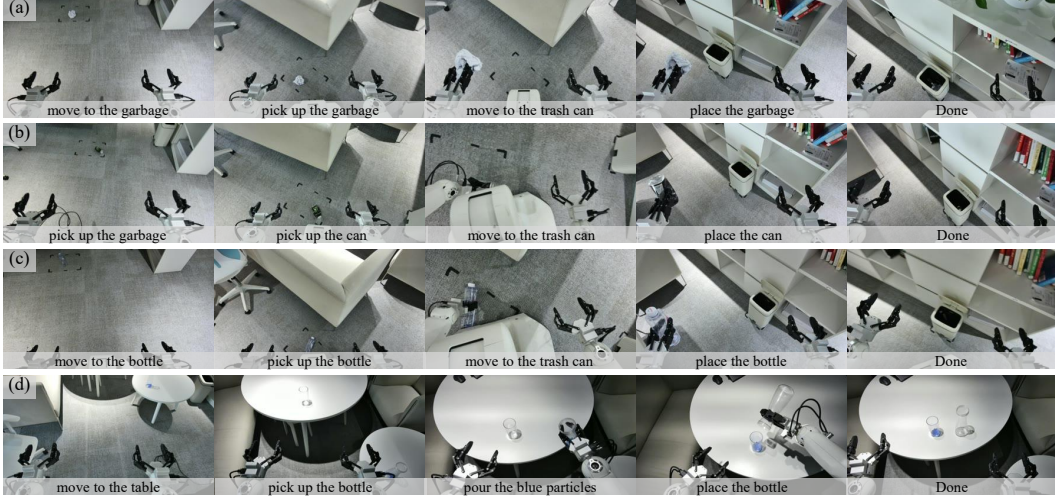


Figure S1: Real-robot rollouts on the Atribot S1, one task per row, with head-camera frames in temporal order (left to right) and the executed skill labelled under each frame. **(a)** trash-garbage: pick up the garbage and place it in the trash can. **(b)** trash-can: pick up the can and place it in the trash can. **(c)** trash-bottle: pick up the bottle and place it in the trash can. **(d)** pour-blue: pick up the bottle and pour the blue particles into the cup.

C Implementation Details of Real-Robot Experiments

C.1 Real-Robot Environment and Embodiment

Platform and Cameras. The real-robot platform is the Atribot S1 [35], a mobile bimanual manipulator with an omnidirectional wheeled base (3 DoF), a 4-DoF articulated torso, two 7-DoF arms, and a 2-DoF head (23 DoF in total); each arm ends in a parallel-jaw gripper rated to a 5 kg payload. The onboard RGB-D suite comprises a head camera (stereo RGB plus an Orbbec Femto Bolt) as the primary scene view, two wrist-mounted Intel RealSense D401 cameras for active-chunk close-ups, and a chest-mounted Orbbec Gemini 335; all stream at 30 Hz. Policy actions are issued at 20 Hz and tracked by the robot’s onboard whole-body controller.

Teleoperation Interface. Demonstrations are collected with the S1’s native VR teleoperation system [35]: a Meta Quest 3S headset with handheld controllers drives the robot through paired first-person (fine manipulation) and third-person (whole-body, low-latency) views. The operator commands per-arm end-effector delta poses in an egocentric frame together with base, torso, head, and gripper targets—a 34-dimensional action with an $SO(3)$ rotation parameterisation, streamed at 100 Hz, which is exactly the action space the Skill Executor’s skill policies predict at deployment. Demonstrations are processed through the same data curation pipeline used in simulation.

Evaluation. 10 trials per task per method, evaluated under a fixed scene-reset protocol; success criteria mirror BDDL goals. Operator interventions are logged for the HIC metric; an intervention is required only on hard physical safety violations. Figure S1 shows a time-ordered rollout of each real-robot task.

C.2 Real-Robot Evaluated Tasks and Subtask Sequences

Table S5 details the natural-language instructions for the four evaluated real-robot tasks, alongside their abbreviated names used throughout the text. During an episode, these instruction strings are passed directly into the *global_goal* field of the Task Planner without modification. Subsequently, the table also outlines the verbatim per-task subtask sequences used to train the per-skill policies of

the Skill Executor in the real world. Each line corresponds to a single segment tagged with one of the canonical skills (`move_to`, `pick_up`, `place_in`, `place_on`, `pour`).

Table S5: Real-robot tasks and their corresponding subtask sequences. The prompt column is used identically as the Task Planner’s deployment prompt and as the per-skill training prompt. The full subtask sequence for each task is given verbatim.

Task ID	Prompt	Subtask Sequence (verbatim)
trash-garbage	Pick up the garbage and place it in the trash can.	1. move to the garbage 2. pick up the garbage 3. move to the trash can while carrying the garbage 4. place the garbage in the trash can
trash-bottle	Pick up the bottle and place it in the trash can.	1. move to the bottle 2. pick up the bottle 3. move to the trash can while carrying the bottle 4. place the bottle in the trash can
trash-can	Pick up the can and place it in the trash can.	1. move to the can 2. pick up the can 3. move to the trash can while carrying the can 4. place the can in the trash can
pour-blue	Pick up the bottle containing blue particles and pour the blue particles into the cup.	1. move to the table 2. pick up the bottle containing blue particles 3. pour the blue particles into the cup 4. place the bottle on the table

C.3 Skill Executor Policy Training

The Skill Executor is a single $\pi_{0.5}$ [3] model that exposes one action expert per canonical skill behind a shared PaliGemma (gemma_2b) backbone; each expert is a gemma_300m head. All experts are initialised from the public $\pi_{0.5}$ base checkpoint and trained on the skill-only segment clips emitted by the Data Curator pipeline. Table S6 reports the training configuration. On the Astribot S1, the Skill Executor is a single five-skill model (`move_to`, `pick_up`, `place_in`, `place_on`, `pour`) trained on the combined trash and pour-blue skill clips. Real-robot runs freeze the vision backbone, use a per-timestamp action normaliser, and fit on $4 \times$ A100 GPUs.

D Additional Quantitative Results and Analysis

This section reports four quantitative analyses that complement the main experiments of Section 5 of the main paper but were moved out of the main body for space: a component-level ablation of the Skill Executor’s per-skill policy heads, the wall-clock cost of the Data Curator loop relative to manual annotation, the per-episode runtime cost of the agentic loop relative to the end-to-end baseline, and a detailed per-subtask success rate analysis.

D.1 Per-skill Policy Heads inside the Skill Executor

The Skill Executor routes each skill to its own diffusion-policy head behind a uniform interface (Section 3.1 of the main paper); here the two low-sample skill pairs are merged—`open/close` share one head and `place_in/place_on` share one head—though each could be split into separate heads given more data. To isolate this design choice from the underlying VLA backbone, we hold the backbone fixed to $\pi_{0.5}$ [3] and compare three weight configurations on `task-0000` (*push-radio*) over 10 trails: **one** $\pi_{0.5}$ **with a single shared head**; **one** $\pi_{0.5}$ **with the per-skill heads above**; and **four separate** $\pi_{0.5}$ **checkpoints, one finetuned per skill (four model copies)**.

All three configurations are finetuned on the same demonstrations for matched steps and compute, so any difference is attributable to the head/checkpoint routing alone (Table S7). The per-skill-head

Table S6: Training hyperparameters for the per-skill $\pi_{0.5}$ fine-tuning on the real robot (Astribot S1).

Setting	Real robot (S1)
Precision	bfloat16
Batch size	240
Training steps	20,000
Warmup steps	100
Learning rate	5×10^{-5}

Table S7: Ablation of per-skill policy heads within the Skill Executor on task-0000 (*push-radio*) across 10 trails. All three configurations utilize the $\pi_{0.5}$ [3] backbone and identical demonstration data; the variation lies solely in the routing mechanism.

Skill Executor Weight Configuration	Success Rate (%)
One $\pi_{0.5}$, one shared head	40
One $\pi_{0.5}$, per-skill heads	70
Four separate $\pi_{0.5}$ checkpoints (one per atomic skill)	70

configuration matches four separate checkpoints at a fraction of the parameter and storage cost, while a single shared head trails badly—supporting the per-skill-head interface used in the full system.

D.2 Data-collection Wall-clock Cost versus Manual Annotation

We compare the *data-processing* step that turns a raw demo into a usable per-skill clip (training excluded), starting both sources from the same teleoperated demos at equivalent clip quality. Manual processing is charged at the demo played back at $2\times$ speed (~ 2.2 min per clip); MobiAgent’s outer loop is charged at its measured end-to-end API wall-clock on the same demos. At 800 clips this is a $\sim 35\times$ reduction (Figure S2).

D.3 Per-episode Runtime

Table S8 compares MobiAgent’s per-episode runtime against the $\pi_{0.5}$ -TA end-to-end baseline [15] on successful rollouts. For MobiAgent, the total runtime comprises the physical action execution time (policy inference plus physics stepping) and the VLM API time. While MobiAgent’s action execution time is moderately higher than the baseline due to running finer-grained skill chunks with per-chunk verification, the primary bottleneck is the VLM API overhead, which is the necessary cost of the closed-loop reasoning that drives its success-rate and robustness gains (Section 5 of the main paper). To mitigate this latency in future deployments, potential solutions include asynchronous processing, where VLM API calls for verification and high-level planning are executed concurrently with the robot’s physical actions, effectively hiding the API latency and significantly reducing the total per-episode runtime.

Table S8: Comparison of per-episode runtime (in seconds) for successful rollouts across the four BEHAVIOR-1K tasks. For MobiAgent, the total runtime is the sum of the action execution time (policy inference and physics stepping, excluding rendering) and the VLM API time. The $\pi_{0.5}$ -TA baseline operates end-to-end without API calls.

Task	$\pi_{0.5}$ -TA	MobiAgent (ours)		
	Total Runtime	Action Exec.	API	Total Runtime
push-radio	60	81	98	179
dispose-trash	249	334	381	715
stack-storage	525	707	747	1454
fetch-beer	538	764	792	1556

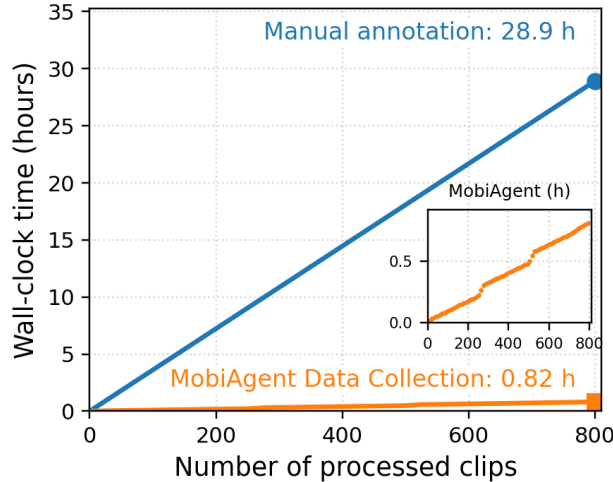


Figure S2: Data-processing wall-clock time vs. number of processed clips across all four tasks. At 800 clips, manual annotation requires ~ 28.9 h compared to MobiAgent’s ~ 0.82 h (a $\sim 35\times$ speedup).

Table S9: Per-skill success rates (%) of the Skill Executor in both simulation and real-world deployments, evaluated on instances successfully reached during execution. The “—” symbol denotes skills that are exclusive to one domain (e.g., open/close are simulation-only, whereas pour is real-world-only).

Atomic Skill	Simulation SR (%)	Real SR (%)
move_to	100.00	100.00
pick_up	88.57	70.00
place_in	75.00	94.73
place_on	100.00	100.00
open	86.67	—
close	100.00	—
pour	—	50.00

D.4 Per-subtask Success Rate

Table S9 reports the Skill Executor’s per-subtask success rate by canonical skill, in simulation and on the real robot, helping to localize where the closed loop’s residual failures concentrate.

E Failure Modes and Analysis

To better understand the limitations of our system, we categorize the observed execution errors into four distinct failure modes (FM1–FM4), each mapping to specific operational bottlenecks. Specifically, **FM1 (Reflection Critic False Positives)** involves high-level evaluation errors where the critic incorrectly assumes a subtask is complete (often due to occlusion or timing issues), causing downstream failures. **FM2 (Bimanual-Coordination Failures)** captures synchronization breakdowns in tightly coupled two-arm tasks, arising because the arms are not explicitly conditioned on each other’s states. **FM3 (Inter-skill Handoff Pose Mismatch)** addresses terminal-pose misalignments during sequential skill transitions, where an off-distribution ending pose of one skill derails the execution of the next. Finally, **FM4 (Low-level Perceptual Limits)** highlights sensing and representation shortcomings when the system deals with complex spatial layouts or transparent objects.

E.1 FM1: Reflection Critic False Positives

A false positive occurs when the Reflection Critic incorrectly evaluates an incomplete subtask as complete, causing the episode to advance based on a flawed premise. Although the critic’s conservative design makes these errors rarer than false negatives, they are significantly more costly because the initial error cascades through all downstream subtasks. A typical manifestation of this issue is a *timing* error, such as terminating a `move_to` action prematurely. As illustrated in Figure S3, the critic erroneously approves the robot’s approach to the trash can while it is still slightly out of range. Consequently, the subsequent `pick_up` action is physically impossible to execute; the robot repeatedly attempts to grasp the can until its retry budget is exhausted, leaving the object untouched on the floor. Fortunately, the system’s closed-loop architecture provides a natural fallback: the inevitable failure of the downstream subtask (e.g., the failed `pick_up`) will eventually be detected by the Reflection Critic, which will then trigger a re-planning phase to recover from the stalled state.



Figure S3: FM1: Reflection Critic completion-timing error on `dispose-trash` (head-camera frames, left to right). Having accepted the `move_to` while the robot was still too far, the subsequent `pick_up` reaches repeatedly but cannot grasp the out-of-range can, which stays on the floor until the retry budget is exhausted.

E.2 FM2: Bimanual-Coordination Failures

Several subtasks require coordinated bimanual actions. However, because the Skill Executor drives both arms from a single head without explicitly conditioning on the partner arm’s state, tightly coupled two-arm motions often suffer from poor synchronization. As illustrated in Figure S4, three recurring failure modes highlight this limitation: (a) In `fetch-beer`, the robot must hold one bottle in one gripper while retrieving a second from the refrigerator. The lack of coordination often causes the reaching arm to knock over the second bottle rather than grasping it cleanly. (b) In `push-radio`, the picking arm grasps the radio in an arbitrary, convenient orientation, which frequently leaves the power button facing away from the pressing arm. This necessitates a fragile in-hand reorientation or hand-over, making it a common source of dropped or misaligned radios. (c) In `stack-storage`, the storage container is too large for a single gripper and requires a simultaneous bimanual grasp. When only one gripper secures it, the container is lifted unevenly, which subsequently derails the carry (`move_to`) and stacking (`place_on`) subtasks. In future work, this limitation could potentially be addressed by introducing more constrained, arm-specific subtask descriptions and generating corresponding targeted training data to better supervise tightly coupled bimanual coordination.

E.3 FM3: Inter-skill Transition and Pose Mismatch

A primary challenge in modular policy execution lies in the *skill transition* phase. While training per-skill policies in isolation significantly improves learning efficiency, it inherently assumes a roughly canonical starting configuration for each skill. Consequently, during the transition between skills, if the preceding skill achieves its functional goal but leaves the manipulator in an out-of-distribution (OOD) pose, the subsequent skill inherits this unfamiliar state and may fail despite no explicit error occurring upstream. Figure S5 illustrates this transition bottleneck in a `dispose-trash` episode: the `pick_up` skill successfully grasps the soda can (top row) but terminates in an awkward pose, raised and rotated away from the bin. The subsequent `place_in` skill (bottom row) initializes from this misaligned configuration and fails to navigate the can over the bin opening. Because the Reflec-

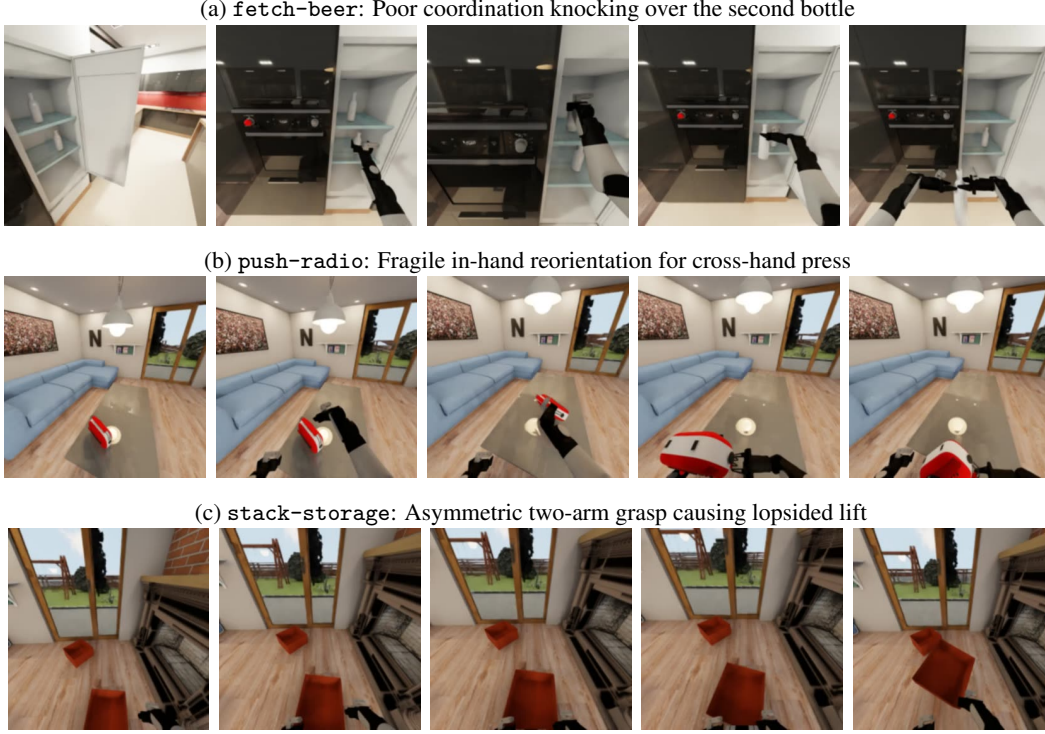


Figure S4: FM2: Bimanual-coordination failures across three tasks. (a) The two arms are poorly coordinated, knocking over the target. (b) The picking arm must flip the radio in-hand, frequently dropping it. (c) Only one gripper secures the large container, causing it to tilt and derailing subsequent stacking.



Figure S5: FM3: Inter-skill handoff pose mismatch on `dispose-trash` (head-camera frames, left to right). The `pick_up` grasps the can off the floor and succeeds, but ends with the arm raised and rotated away from the carried bin; the next `place_in` inherits this pose, cannot bring the can over the bin opening, and the placement fails.

tion Critic currently evaluates each subtask strictly against its isolated goal, the successful `pick_up` is accepted, and the kinematic deviation only manifests during the transition to the next step.

To address this transition bottleneck in future work, the agent framework could be enhanced with transition-aware mechanisms. At the system level, we could dynamically insert a transitional `reset_pose` or `reorient` skill between highly misaligned subtasks to bridge the kinematic gap. Alternatively, the Reflection Critic could be augmented to evaluate not just subtask completion, but also the kinematic viability of the handoff state for the upcoming transition. At the policy level, injecting starting-state noise during skill training would also improve the downstream policy’s robustness to imperfect transitions.

E.4 FM4: Low-level Perceptual Limits

The shared per-skill policies inherit the perceptual limitations of the underlying action expert, which prominently surface in two real-robot tasks (see Figure S6). In `trash-bottle` (a), inadequate spatial and metric perception causes the policy to misjudge the relative position of the trash bin, leading the robot to release the bottle just outside the rim so that it falls onto the floor. In `pour-blue` (b), the clear glass is difficult to localize against the bright table surface; consequently, the policy misaligns the bottle and pours the liquid next to the glass rather than into it. Both failures stem fundamentally from the sensing and representational bottlenecks of the current action expert. Moving forward, these limitations could be addressed by enhancing the dedicated perception module (e.g., incorporating explicit 3D representations or depth sensing). Furthermore, owing to the modular design of our agent framework, any future enhancements in state-of-the-art Vision-Language-Action (VLA) policies can be seamlessly integrated, allowing the system to effortlessly inherit stronger perceptual and spatial reasoning capabilities without requiring architectural overhauls.

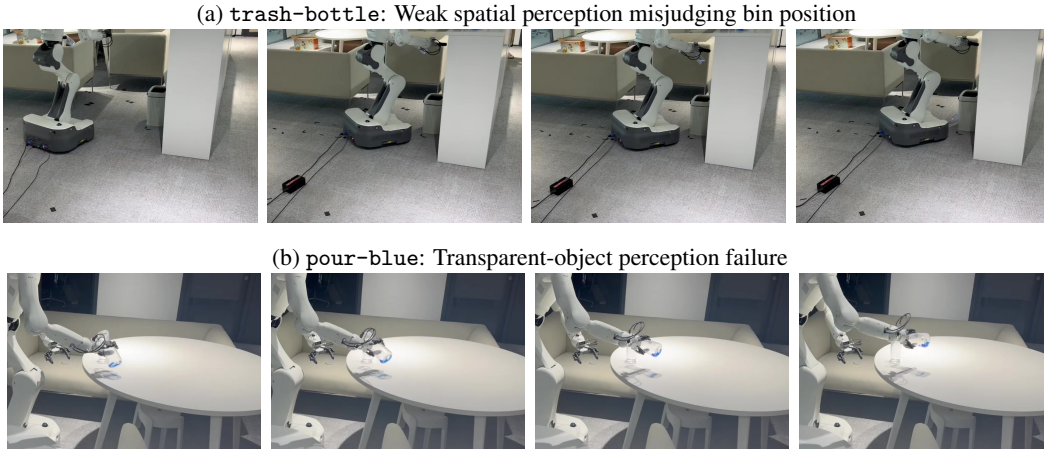


Figure S6: FM4: Low-level perceptual limits on the real robot. (a) The policy misjudges the bin’s spatial configuration, dropping the bottle on the floor instead of inside. (b) The clear glass is hard to localise, causing the liquid to be poured onto the table beside the glass.

F Comparison with Other Agentic Frameworks

F.1 Comparison with CaP-X

CaP-X [27] is a concurrent Code-as-Policy framework: a coding agent (a VLM) synthesises an executable program that composes perception primitives (open-vocabulary segmentation and pointing) and control primitives to drive the robot. On its mobile embodiment the only manipulation primitive is a grasp: `sample_grasp_pose` runs a learned 6-DoF grasp-pose network (Contact-GraspNet [38]) on the segmented object’s depth point cloud, falling back to top-down / bounding-box heuristic grasps when the network returns none, and the program tries the returned candidates one at a time (motion-plan to the pre-grasp pose, close the gripper, lift) until one holds. Its mobile primitive set is thus just navigation, this one-shot grasp, and gripper open/close: it exposes no `place`, `open/close`, or `press` primitive, so it cannot release into a receptacle, stack an object, or actuate an articulated object, and therefore cannot carry a long-horizon task to completion. CaP-X’s own BEHAVIOR evaluation reflects this: it reduces its two mobile tasks to “pick up the radio” and “pick up a soda can” (navigate + grasp only). We accordingly compare on exactly the navigate-and-grasp portion of those two tasks (`push-radio` and `dispose-trash`). Following CaP-X’s own protocol we report navigation success (the robot reaches within ~ 1 m of the target) and task (grasp) success separately, over 10 trials per task under matched observations and embodiment (Table S10). Our 10 trials use *held-out test scenes* unseen in training, whereas CaP-X’s original 25-scene protocol drew from the

Table S10: Comparison with CaP-X [27] on the two tasks within its navigate-and-grasp repertoire (10 trials each, held-out test scenes, matched observations and embodiment). Navigation and task (including navigation) success are reported separately (%), following CaP-X’s protocol.

Task	CaP-X [27]		MobiAgent (ours)	
	Navigation SR	Task SR	Navigation SR	Task SR
push-radio	100	20	100	90
dispose-trash	90	80	100	100

Table S11: RoboClaw’s planner decomposition of the four evaluated tasks (verbatim subtask prompts from its run logs). Each subtask is a coarse composite that already bundles several of MobiAgent’s atomic skills (e.g. `move_to` + `pick_up` + `place_in`).

Task	RoboClaw subtask prompts (verbatim)
push-radio (2)	1. Move both arms clear, visually locate the radio on the coffee table in the living room, and position yourself so one gripper can safely reach the radio power control. 2. Use the most suitable gripper to press or toggle the radio’s power control on the coffee table until the radio turns on.
dispose-trash (3)	1. Pick up one soda can from the living room, move to the kitchen trash can, and drop the can into the trash. Finish with empty grippers and do not pick a second can in the same run. 2. Pick up one remaining soda can from the living room, move to the kitchen trash can, and drop the can into the trash. Finish with empty grippers and do not pick another can in the same run. 3. Pick up the last remaining soda can from the living room, move to the kitchen trash can, and drop the can into the trash. Finish with empty grippers.
stack-storage (2)	1. Pick up one storage box in the living room, carry or move it to the garage, and place it stably on the garage floor where it can serve as the base for stacking. 2. Pick up the remaining storage box in the living room, move it to the garage, align it above the first box already on the garage floor, and place it stably on top.
fetch-beer (5)	1. Open the fridge door, grasp one bottle of beer from inside the fridge, and take it out securely. 2. Place the beer bottle you are holding onto the coffee table in a stable upright position. 3. Return to the open fridge, grasp a second bottle of beer from inside the fridge, and take it out securely. 4. Place the second beer bottle onto the coffee table in a stable upright position near the first bottle. 5. Close the fridge door fully without disturbing the bottles on the coffee table.

training split; since MobiAgent is learned and CaP-X is training-free, this makes the comparison conservative for us. CaP-X navigates reliably but grasping is its bottleneck; MobiAgent both navigates and grasps reliably and additionally executes the subsequent `press/place_in` steps that complete each task, which CaP-X cannot attempt.

F.2 Comparison with RoboClaw

We also examine RoboClaw [13], a concurrent agentic framework designed for scalable long-horizon tasks. RoboClaw unifies data collection, policy learning, and task execution under a single VLM-driven controller, utilizing Entangled Action Pairs (EAP) to form self-resetting learned policy primitives. Despite its robust multi-policy orchestration, it cannot be applied directly to our skill-conditioned policy due to a fundamental architectural mismatch. RoboClaw’s planner decomposes tasks into *coarse*, monolithic subtasks (e.g., an entire “pick → navigate → place” composite issued as a single prompt; see Table S11). Because a single RoboClaw instruction spans multiple MobiAgent’s atomic skills simultaneously, its plans cannot be routed to our skill-conditioned experts

Table S12: Comparison of planning paradigms. RoboClaw employs a static, plan-then-execute approach with conditional recovery, whereas MobiAgent continuously re-plans at every step for dynamic adaptation.

Execution Phase	RoboClaw	MobiAgent (ours)
Task Initialization	Generates the complete, ordered plan upfront.	Invokes reactive Task Planner for the immediate subtask (full sketch optional).
Normal Execution	Executes the fixed plan sequentially without deviation.	Re-queries the Task Planner at each step; continuously adapts to observations.
Error Recovery	Triggers conditional re-planning via an explicit <code>replan</code> action.	Inherently recovers by re-evaluating the current state at the next step.

without finer re-segmentation. Consequently, we report this architectural divergence rather than a direct head-to-head quantitative comparison.

Static plan-then-execute vs. dynamic re-planning. The two frameworks also differ fundamentally in *when* planning occurs (Table S12). While RoboClaw’s VLM dynamically orchestrates its EAP primitives, at the macro-task level it employs a static, plan-then-execute approach: it generates the full ordered plan upfront and executes it unchanged, only triggering a re-plan if a subtask’s success check fails. In contrast, MobiAgent dynamically re-invokes the Task Planner at every step to determine the next subtask based on the current observation. This continuous adaptation, rather than relying solely on failure-triggered corrections, is the architectural driver behind MobiAgent’s superior robustness.

Robustness to instruction paraphrasing. A reliable long-horizon system should produce consistent task decompositions for semantically equivalent instructions. To evaluate this, we hold the task goal fixed and reword its instruction into three distinct paraphrases (Table S14). We feed each phrasing to both planners using the same GPT-5.4 backbone and starting frame, and record the number and structure of the emitted subtasks. Table S15 reproduces every emitted plan verbatim, while Table S13 summarises the subtask counts.

The results highlight a stark contrast in paraphrase robustness. As quantified in Table S13, MobiAgent emits a near-constant number of subtasks across different phrasings (with the only minor deviation being a redundant per-bottle cycle elicited by one `fetch-beer` wording). Its atomic steps remain fixed in number and order; only the object wording adapts to the instruction (e.g., *trash can* vs. *kitchen bin*, *fridge* vs. *refrigerator*). Conversely, RoboClaw’s free-form subtasks fluctuate widely in both count and granularity across all tasks, particularly on `dispose-trash`.

This discrepancy is rooted in the architectural design. MobiAgent’s Task Planner selects each subtask from a fixed catalogue of atomic skills. Thus, the granularity is constrained by the skill set rather than the instruction’s surface form—rewording alters the vocabulary *within* a step, but not the overall step count. RoboClaw’s free-form generation, however, produces verbose and inconsistent strings that vary with the phrasing. If routed to a policy, these would be far out-of-distribution compared to the short, canonical prompts used during training. By conditioning every expert on a short, fixed skill string (e.g., `pick up bottle of beer from fridge`), MobiAgent ensures its plans remain paraphrase-robust and its policy prompts strictly in-distribution.

Table S13: Instruction robustness: Number of subtasks emitted by each planner for the three semantically equivalent phrasings (v0, v1, v2) detailed in Table S14. The spread $\Delta = \max - \min$ quantifies consistency across paraphrases. Both planners utilize the same GPT-5.4 backbone and starting frame to isolate the evaluation to the planning decomposition. paraphrase-robust planner yields a consistent subtask count (i.e., a near-zero Δ).

Task	RoboClaw				MobiAgent (ours)			
	v0	v1	v2	Δ	v0	v1	v2	Δ
push-radio	3	2	3	1	4	4	4	0
dispose-trash	6	12	3	9	12	12	12	0
stack-storage	4	2	2	2	10	9	10	1
fetch-beer	5	4	4	1	7	7	10	3
Mean Δ	3.25				1.00			

Table S14: Instruction paraphrases utilized in the robustness evaluation. For each of the four BEHAVIOR-1K tasks, the underlying goal remains fixed while the instruction is reworded into three semantically equivalent variants (v0, v1, v2). Both planners receive the exact same prompt string for a given variant.

Task	Variant	Instruction
push-radio	v0	Turn on the radio sitting on the coffee table in the living room.
	v1	Switch on the radio that is on the living-room coffee table.
	v2	Power up the radio resting on the coffee table in the living room.
dispose-trash	v0	There are three cans of soda in the living room. Pick up each can and throw it into the trash can in the kitchen, one at a time, until all three cans are in the kitchen trash can.
	v1	Take the three soda cans from the living room and drop them, one by one, into the kitchen trash can.
	v2	Clear all three soda cans out of the living room by putting each one into the kitchen bin, handling them one at a time.
stack-storage	v0	Move the two storage boxes from the living room to the garage. Place the first box on the garage floor and stack the second box on top of it.
	v1	Carry both storage boxes from the living room into the garage; set one box down on the garage floor and put the other box on top of it.
	v2	Relocate the two storage boxes out of the living room into the garage, leaving the first on the floor and stacking the second over it.
fetch-beer	v0	Take two bottles of beer from the fridge to the coffee table, then close the fridge door.
	v1	Bring two beer bottles out of the fridge over to the coffee table, and afterwards shut the fridge door.
	v2	Move two bottles of beer from the refrigerator onto the coffee table, then close the refrigerator.

Table S15: Verbatim emitted plans for the instruction-robustness study. This table presents every subtask produced by each planner across the four BEHAVIOR-1K tasks and their three semantically equivalent phrasings (v0, v1, v2) detailed in Table S14.

Variant	RoboClaw	MobiAgent (ours)
push-radio (task-0000)		
v0	1. Move to the living room coffee table and position yourself within reach of the radio while keeping a safe manipulation distance. 2. Visually align an arm and gripper with the radio's power button or power switch so the control can be pressed or toggled cleanly without knocking the radio over. 3. Press or toggle the radio's power control once to turn the radio on, then release contact without moving the radio significantly.	1. move to radio 2. pick up radio from coffee table 3. press radio 4. place radio on coffee table
v1	1. Find the radio on the living-room coffee table, identify its power button or switch, and move into a stable pre-press position with a clear approach to the control without activating it yet. 2. Use a careful precise motion to press or toggle the radio's power control on the living-room coffee table and turn the radio on. Stop after the control has been actuated once clearly.	1. move to radio 2. pick up radio from living-room coffee table 3. press radio 4. place radio on living-room coffee table
v2	1. Approach the radio on the coffee table in the living room and position your gripper in front of its power control with a stable, collision-free pose ready to actuate it. 2. Actuate the radio's power control on the coffee table to turn the radio on. Press the power button firmly once or rotate the power knob only as much as needed to switch the device from off to on, then release safely. 3. Hold position briefly, observe the radio after actuation, and make any minimal corrective re-press only if needed so the radio is left powered on.	1. move to radio 2. pick up radio from coffee table 3. press radio 4. place radio on coffee table

(Table S15, continued)

Variant	RoboClaw	MobiAgent (ours)
dispose-trash (task-0001)		
v0	1. In the living room, locate one soda can, approach it safely, and grasp it securely with a stable hold for transport. 2. Carry the grasped soda can from the living room to the kitchen trash can, position it over the trash opening, and release it so the can lands inside the trash can. 3. Return attention to the living room, locate a second soda can that is not yet disposed, approach it safely, and grasp it securely with a stable hold for transport. 4. Carry the second grasped soda can from the living room to the kitchen trash can, position it over the trash opening, and release it so the can lands inside the trash can. 5. Locate the last remaining soda can in the living room, approach it safely, and grasp it securely for transport without disturbing the environment unnecessarily. 6. Carry the last grasped soda can to the kitchen trash can, align over the trash opening, and release it so the can lands fully inside the trash can.	1. move to trash can 2. pick up trash can from floor 3. move to can of soda 4. pick up can of soda from floor 5. place can of soda in trash can 6. move to can of soda 7. pick up can of soda from floor 8. place can of soda in trash can 9. move to can of soda 10. pick up can of soda from floor 11. place can of soda in trash can 12. place trash can on floor
v1	1. Navigate to the first soda can in the living room and stop in a stable pose that allows grasping. 2. Pick up the first soda can from the living room with a stable grasp and lift it clear of nearby surfaces. 3. Carry the soda can to the kitchen trash can and align it above the trash opening without colliding with the bin or surroundings. 4. Release the soda can so it falls into the kitchen trash can, then retract the gripper without pulling the can back out. 5. Navigate back to the living room and stop in a stable pose near the second remaining soda can for grasping. 6. Pick up the second soda can from the living room with a stable grasp and lift it clear of nearby surfaces. 7. Carry the soda can to the kitchen trash can and align it above the trash opening without colliding with the bin or surroundings. 8. Release the soda can so it falls into the kitchen trash can, then retract the gripper without pulling the can back out. 9. Navigate back to the living room and stop in a stable pose near the third remaining soda can for grasping. 10. Pick up the third soda can from the living room with a stable grasp and lift it clear of nearby surfaces. 11. Carry the soda can to the kitchen trash can and align it above the trash opening without colliding with the bin or surroundings. 12. Release the soda can so it falls into the kitchen trash can, then retract the gripper without pulling the can back out.	1. move to trash can 2. pick up trash can from floor 3. move to soda can 4. pick up soda can from floor 5. place soda can in trash can 6. move to soda can 7. pick up soda can from floor 8. place soda can in trash can 9. move to soda can 10. pick up soda can from floor 11. place soda can in trash can 12. place trash can on floor

(Table S15, continued)

Variant	RoboClaw	MobiAgent (ours)
v2	1. Find one soda can in the living room, pick it up securely, carry it to the kitchen bin, and place it fully inside the bin. 2. Find the next remaining soda can in the living room, pick it up securely, carry it to the kitchen bin, and place it fully inside the bin without disturbing cans already deposited. 3. Find the last remaining soda can in the living room, pick it up securely, carry it to the kitchen bin, and place it fully inside the bin, leaving no soda cans remaining in the living room.	1. move to kitchen bin 2. pick up kitchen bin from floor 3. move to soda can 4. pick up soda can from floor 5. place soda can in kitchen bin 6. move to soda can 7. pick up soda can from floor 8. place soda can in kitchen bin 9. move to soda can 10. pick up soda can from floor 11. place soda can in kitchen bin 12. place kitchen bin on floor
stack-storage (task-0016)		
v0	1. Identify one storage box in the living room, reach to it, grasp it securely with both arms if needed, and lift it into a stable carrying pose. 2. Carry the grasped storage box from the living room to the garage and place it flat and stably on the garage floor at a location suitable for stacking a second box on top. 3. Return to the remaining storage box in the living room, grasp it securely with both arms if needed, and lift it into a stable carrying pose. 4. Carry the second storage box from the living room to the garage, align it above the first box already on the garage floor, and place it centered and stably on top of the first box before releasing.	1. move to door 2. open door 3. move to storage box 4. pick up storage box from floor 5. move to floor 6. place storage box on floor 7. move to storage box 8. pick up storage box from floor 9. move to storage box 10. place storage box on storage box
v1	1. Pick up one storage box from the living room, carry it into the garage, and place it stably on the garage floor leaving enough free space above it for stacking a second box. 2. Pick up the remaining storage box from the living room, carry it into the garage, align it above the box already on the garage floor, and place it stably on top without tipping or sliding.	1. move to door 2. open door 3. move to storage box 4. pick up storage box from floor 5. move to storage box 6. pick up storage box from floor 7. move to floor 8. place storage box on floor 9. place storage box on storage box
v2	1. Pick up one storage box in the living room, carry it into the garage, and place it flat and stably on the garage floor where it can serve as the base for stacking a second box. Release only after the box is stable and not touching obstacles. 2. Pick up the remaining storage box in the living room, carry it into the garage, align it over the first box, and place it stably on top of that box. Release only when the upper box is balanced and fully supported by the lower box.	1. move to door 2. open door 3. move to storage box 4. pick up storage box from floor 5. move to floor 6. place storage box on floor 7. move to storage box 8. pick up storage box from floor 9. move to storage box 10. place storage box on storage box

(Table S15, continued)

Variant	RoboClaw	MobiAgent (ours)
fetch-beer (task-0017)		
v0	1. Open the fridge door, grasp one bottle of beer from inside the fridge, and take it out securely. 2. Place the beer bottle you are holding onto the coffee table in a stable upright position. 3. Return to the open fridge, grasp a second bottle of beer from inside the fridge, and take it out securely. 4. Place the second beer bottle onto the coffee table in a stable upright position near the first bottle. 5. Close the fridge door fully without disturbing the bottles on the coffee table.	1. move to fridge 2. open fridge 3. pick up bottle of beer from fridge 4. move to coffee table 5. place bottle of beer on coffee table 6. move to fridge 7. close fridge
v1	1. Open the fridge door wide enough to access the beer bottles inside. 2. Take one beer bottle out of the open fridge and place it upright on the coffee table. 3. Take a second beer bottle out of the open fridge and place it upright on the coffee table next to the first one. 4. Close the fridge door fully.	1. move to fridge 2. open fridge 3. pick up beer bottle from fridge 4. move to coffee table 5. place beer bottle on coffee table 6. move to fridge 7. close fridge
v2	1. Open the refrigerator door enough to access the beer bottles inside. 2. Pick up one bottle of beer from inside the refrigerator, carry it carefully, and place it upright on the coffee table. 3. Pick up a second bottle of beer from inside the refrigerator, carry it carefully, and place it upright on the coffee table next to the first bottle. 4. Close the refrigerator door fully after removing the two beer bottles.	1. move to refrigerator 2. open refrigerator 3. pick up bottle of beer from refrigerator 4. move to coffee table 5. place bottle of beer on coffee table 6. pick up bottle of beer from refrigerator 7. move to coffee table 8. place bottle of beer on coffee table 9. move to refrigerator 10. close refrigerator